



Today's agenda

↳ Bridges

↳ Articulation Point



AlgoPrep



Bridges

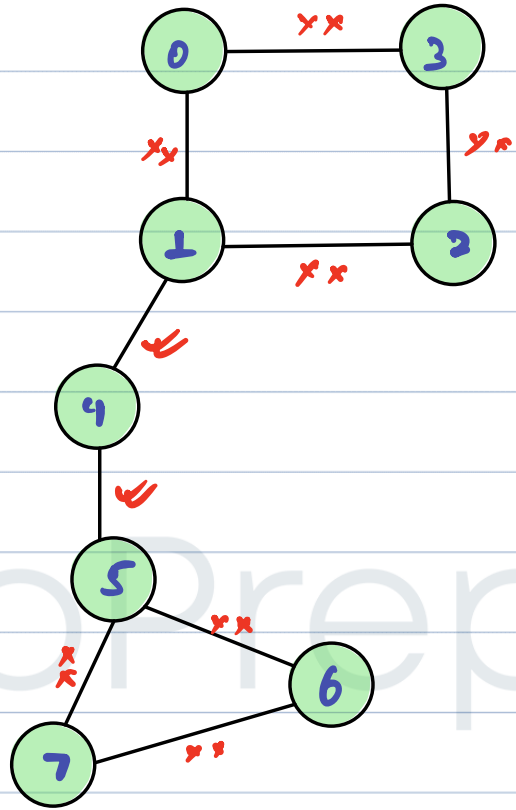
↳ An edge if removed will divide the graph in 2 or more than 2 graph component.

{1, 4} ←
4, 5}

Idea 1

↳ Remove edges one by one and Count the no. of components. Add the valid edges to your ans.

T.C: $O((V+E)*E)$

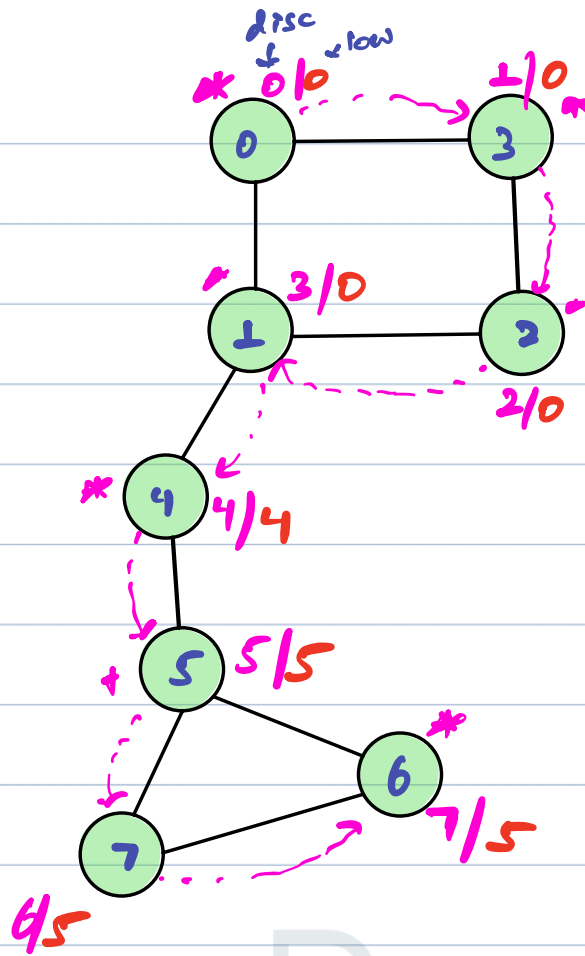


time: 0 1 2 3 4 5 6 7



Idea 2

if $low[nbr] > disc[src]$
you got a bridge



AlgoPrep



//Pseudo code

boolean[] vis;

int[] disc;

int[] low;

int time = 0;

void Bridges (List<List<Integer>> graph, int n, int src, int par){

disc[src] = time;

low[src] = time;

time++;

List<Integer> nbrs = graph.get(src);

T.C: $O(V+E)$

S.C: $O(V)$

for (int nbrs : nbrs) {

if (nbr == par) {

continue;

}

else if (vis[nbr] == false) {

vis[nbr] = true;

Bridges (graph, n, nbr, src);

low[src] = min(low[src], low[nbr]);

if (low[nbr] > disc[src]) { (src, nbr); }

}

else { //vis[nbr] == true

low[src] = min (low[src], disc[nbr]);

}

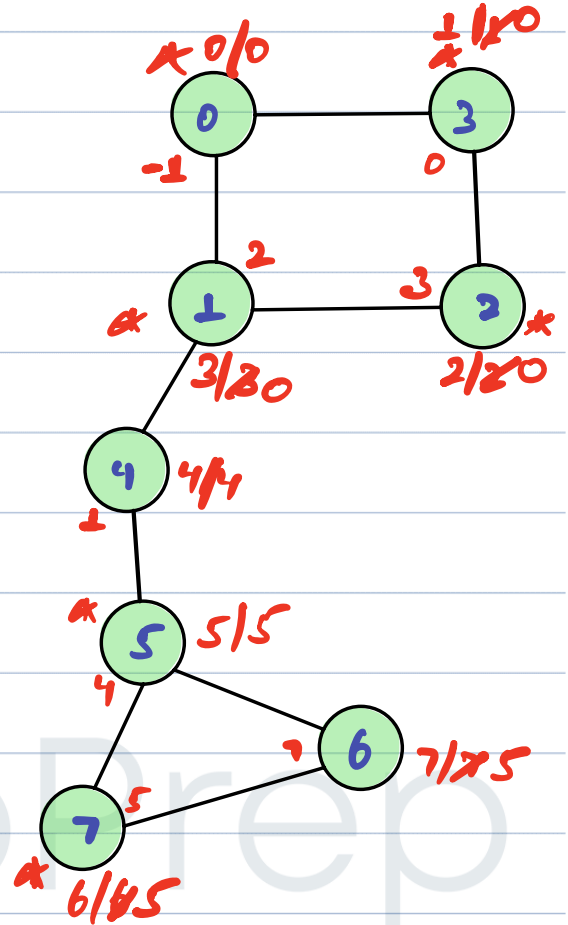
}

}



```
boolean[] vis;  
int[] disc;  
int[] low;  
int time = 0;  
void Bridges(List<List<Integer>> graph, int n, int src, int par){  
    disc[src] = time;  
    low[src] = time;  
    time++;  
    List<Integer> nbs = graph.get(src);  
    for (int nbrs : nbs) {  
        if (nbrs == par) {  
            continue;  
        }  
        else if (vis[nbrs] == false) {  
            vis[nbrs] = true;  
            Bridges(graph, n, nbrs, src);  
            low[src] = min(low[src], low[nbrs]);  
        }  
        else { // vis[nbrs] == true  
            low[src] = min(low[src], disc[nbrs]);  
        }  
    }  
}
```

time: 7 1 2 2 4 8 6 7 8

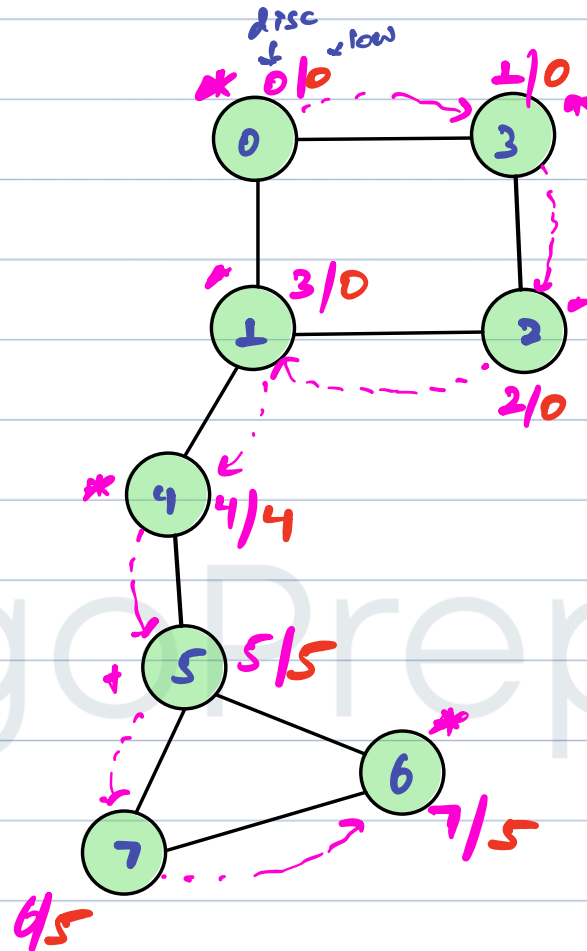




Articulation Point

↳ A node if removed will divide the graph in 2 or more than 2 graph components.

↳ $low[nb] \geq disc[src]$





//Pseudo code

boolean[] vis;

boolean[] ans;

int[] disc;

int[] low;

int time = 0;

void AP (List<List<Integer>> graph, int n, int src, int par) {

disc[src] = time;

low[src] = time;

time++;

List<Integer> nbrs = graph.get(src);

for (int nbr : nbrs) {

if (nbr == par) {

continue;

}

else if (vis[nbr] == false) {

vis[nbr] = true;

Bridges (graph, n, nbr, src);

low[src] = min(low[src], low[nbr]);

if (low[nbr] > disc[src]) { ans[src] = true; }

}

else { // vis[nbr] == true

low[src] = min(low[src], disc[nbr]);

}

}

}

T.C: $O(V+E)$

S.C: $O(V)$



Q) if $nbr == visited$, why not $low[nbr]$.

